

Use of Generative Artificial Intelligence

AI Usage Log Table

Pipeline Phase	SLE Phase	Tool	Usage Type	Prompt Type	# Iter .	Quality	Prompt Excerpt	Accepted / Modified / Rejected
Domain Analysis (P1)	Domain Analysis	Claude Sonnet 3.7	Concept explanation	Zero-shot	2	Good	"Explain the key mechanics of Life is Strange: scenes, choices, butterfly effects, inventory..."	Accepted: provided a solid overview of the game's narrative structure, cross-episode variables and branching endings. Helped bootstrap the domain vocabulary and use case scenarios since neither group member had played the game.
Domain Analysis (P1)	Domain Analysis	Claude Sonnet 3.7 and Gemini Pro	Artifact review	Few-shot iterative	3	Good	"Review these 5 use case scenarios for a StoryFlowDSL. Are the scenarios realistic and do they cover the main domain mechanics?"	Modified: AI suggested adding an explicit cross-episode variable persistence scenario (UC2 plant). We restructured and expanded the scenario descriptions based on the feedback.
Metamodeling (P2)	Abstract syntax	Claude Sonnet 3.7 and Gemini Pro	Artifact review	Few-shot iterative	4	Satisfactory	"Review this Ecore metamodel for a branching narrative DSL. Check multiplicities, missing references, and whether the DiceRule design makes sense..."	Modified: AI flagged that DiceRule had no explicit successTarget/failureTarget on the HLL side, prompting us to clarify the design. Some suggestions about subclassing were rejected; we kept boolean flags as per our design justification.
EVL Validation (P4)	Static semantics	Claude Sonnet 3.7	Debug / error fixing	Iterative with refinement	5	Satisfactory	"This EVL constraint is throwing: 'Feature scenes is not defined for type Story'. Here is the constraint and the metamodel structure..."	Modified: AI correctly identified that scenes are nested under chapters and suggested the correct two-level navigation. The fix was valid after we adapted it to our exact Ecore structure.

ETL Transformation (P5)	Dynamic semantics	Claude Sonnet 3.7	Debug / error fixing	Iterative with refinement	7	Satisfactory	"ETL error: 'Cannot call method equivalent on null' in the Choice → Edge rule. Here is the full rule..."	Accepted (partially): AI correctly diagnosed that .equivalent() was being called before the target rule had run and suggested reordering the rules. The fix worked after we verified the rule execution order in our specific metamodel.
EGL Generation (P6)	Generation	Claude Sonnet 3.7	Code generation	Few-shot iterative	5	Good	"Generate an HTML interactive story from the DarkRoom_pivot. Nodes have id, label, isStart, isAccepting. Edges have guard expression, actions..."	Modified: The overall HTML/JS structure and game loop logic were good and served as the foundation. We rewrote the EGL iteration syntax and adapted the guard evaluation logic to match our exact pivot model structure.
Graphical Editor (Sirius) (P3)	Concrete syntax	Claude Sonnet 3.7 and ChatGPT	Concept explanation	Zero-shot / Few-shot iterative	6	Satisfactory	"Explain how Sirius .odesign works: layers, node mappings, edge mappings, and how to link diagram elements to Ecore classes. How do I configure a NodeCreationTool to create a Scene?"	Modified: AI provided a useful conceptual overview of Sirius diagram descriptions, mapping types, and tool palette configuration. The explanations were accurate at a high level but often referenced older Sirius API versions. We had to cross-reference with the official Sirius documentation to adapt the examples to our exact setup.
Graphical Editor (Sirius) (P3)	Concrete syntax	Claude Sonnet 3.7	Code generation / Debug	Iterative with refinement	5	Satisfactory	"Generate a Java Services class for Sirius that uses reflective EObject access (no generated stubs) to: edit a Condition expression on a Choice, add/remove an Effect, and edit a DiceRule with min/max/successThreshold."	Modified: AI generated a first version of the reflective Services.java using EcoreUtil and eGet/eSet patterns. The reflective approach (avoiding import of generated stubs) was AI's suggestion and proved correct. However, several method calls used wrong feature names and the DiceRule onSuccess/onFailure wiring was missing. We fixed these bugs through iterative testing in the Sirius Workbench.

Formal Verification (P7)	Formal verification	Claude Sonnet 3.7 and ChatGPT	Concept explanation	Zero-shot / Few-shot iterative	5	Satisfactory	"Explain NuSMV module structure for a state machine: how to declare state variables as enums, write ASSIGN init/next blocks, encode boolean guards, and write CTL properties for reachability and deadlock freedom."	Modified: AI gave a solid conceptual introduction to NuSMV MODULE, VAR, ASSIGN, and SPEC blocks. The explanations of CTL operators (EF, AG, EX) were accurate and directly informed our property specifications. The generated .smv examples were syntactically correct when produced in isolation but broke when combined with EGL template generation.
Formal Verification (P7)	Formal verification	Claude Sonnet 3.7	Debug / error fixing	Iterative with refinement	4	Satisfactory	"This NuSMV model is returning: 'type error: next(...) applied to a non-variable'. Here is the ASSIGN block and the VAR declaration. What is wrong?"	Accepted: AI correctly identified that a NEXT() assignment referenced a computed expression rather than a declared state variable, and suggested restructuring the VAR section to introduce an auxiliary state variable. The fix was valid and directly applicable. AI was consistently more useful for diagnosing NuSMV errors than for generating NuSMV from scratch.

Reflection on AI usage

Phases Where AI Was Most Useful

AI proved most useful in three distinct areas of the project:

Domain bootstrapping (Phase 1): Since neither group member had played Life is Strange, AI was invaluable for quickly building a working understanding of the game's narrative mechanics, cross-episode variables, butterfly-effect choices, and branching endings. Rather than spending hours playing or reading wikis, we were able to describe the domain to AI and receive a structured overview that directly informed our use case scenarios. This was a genuine productivity gain with no meaningful risk, as the output was used for domain understanding and not for executable artifacts.

Debugging Epsilon errors (Phases 4, 5 and 6): When EVL, ETL and EGL were producing cryptic error messages, AI was a good first-line diagnostic tool. Provided with a clear error message and the relevant code snippet, it correctly identified the root cause in most cases. This accelerated the debug cycle considerably compared to consulting documentation alone, and it consistently outperformed zero-shot generation on the same tasks.

EGL code generation scaffolding (Phase 6): For the HTML/JS interactive story runner, AI produced a solid structural foundation. The game loop logic, guard evaluation skeleton, and state persistence pattern were all usable. This was AI's strongest generative contribution to the project, reflecting the high representation of HTML and JavaScript in training data.

Phases Where AI Was Less Useful or Produced Incorrect Output

The project confirmed the warning in the specification: LLMs are significantly weaker on niche, low-resource languages. This manifested in three distinct ways.

ETL transformations (Phase 5): This was the most frustrating phase for AI-assisted generation. Across multiple attempts, AI produced ETL that looked plausible but contained consistent hallucinations: invented method names, wrong rule syntax, and incorrect use of `.equivalent()`. The core issue is that ETL (Epsilon Transformation Language) has minimal representation in training data. By iteration 4–5, we had learned more about ETL from fixing the AI's mistakes than from any individual correct output. We ended up writing the rules ourselves.

NuSMV generation via EGL (Phase 7): Similar issues arose with NuSMV. AI demonstrated knowledge of NuSMV syntax in isolation, but combining it with EGL template generation introduced compounding errors: malformed MODULE structures, incorrect ASSIGN blocks, and invalid VAR declarations. After four failed attempts, we wrote the .smv template by hand. AI remained useful for verifying individual CTL formula logic when presented in isolation.

Architectural separation (cross-phase): When asked to generate an HTML interface, AI initially attempted to read data directly from the HLL metamodel (accessing Chapters and Scenes), entirely bypassing the Pivot narrative graph established via ETL. Recognising and correcting this required a thorough understanding of our own architectural decisions — which, in retrospect, made the exercise valuable even if the AI output was wrong.

Ecore metamodel review (Phase 2): AI suggestions for metamodel structure were reasonable but persistently generic. It repeatedly advocated for subclassing over boolean flags, which contradicts our deliberate design decision to keep the metamodel flat and avoid unnecessary class proliferation. Assertively rejecting these suggestions required us to articulate and defend our design rationale, which was itself a useful exercise.

How the Prompting Strategy Evolved

Early in the project we used zero-shot prompts expecting complete, working artifacts. This quickly proved ineffective for Epsilon-specific tasks. Our strategy evolved through two key realisations. First, every prompt involving Epsilon code must include the relevant metamodel structure, class names, attributes, and relationships, explicitly. Without this context, AI consistently invented class names and assumed structural properties that did not exist in our metamodel. Second, we learned to use AI reactively rather than generatively: write code ourselves, execute it, and then paste the error message together with the failing code and ask for diagnosis. This approach played to AI's relative strength (error explanation and targeted correction) rather than its weakness (generating novel Epsilon code from scratch).

The Role of Generative AI in Language Engineering

Our position after this project is that generative AI is both a genuine productivity tool and a real dependency risk in Language Engineering, and the balance depends heavily on which part of the SLE workflow is involved.

For domain analysis and documentation, AI accelerates work without meaningful risk. For code generation in well-represented languages (HTML, JavaScript, Python), AI is a reliable productivity multiplier. For model transformation languages like ETL and EGL, and for formal verification languages like NuSMV, AI is unreliable enough that over-reliance would be actively harmful to learning.

The dependency risk is subtle: AI generates confident-sounding, syntactically plausible code that only fails at runtime. This creates a false sense of progress.

Recommendation for Future Students

We would recommend AI usage to future students, but with the following specific guidance:

- Use AI extensively for domain analysis, use case formulation, and understanding reference systems you are not familiar with.
- Use AI for debugging: paste error messages and code snippets. It diagnoses Epsilon errors reasonably well when given sufficient context.
- Do not attempt to generate complete ETL, EVL, or EGL files from scratch using AI and then fix them. The hallucination rate on niche languages makes this strategy less efficient than writing the rules directly from the documentation.
- When AI output is wrong, treat it as a learning opportunity: understanding why it is wrong requires understanding the correct semantics. Several of our most effective learning moments came from diagnosing AI mistakes.